



# 启动链、GRUB、Target 与系统恢复

从固件到 PID 1：先判断启动阶段与真实根，再完成最小修复和重启验收。

对象模型操作语义验证诊断经典任务

大字号阅读版

# 本章阅读导航

先抓住一条主线：启动不是一个瞬间，而是 firmware、GRUB、kernel、initramfs、真实根和 systemd 逐层移交控制权。恢复时先判断当前阶段和路径语境，再区分一次性状态、持久配置与最终功能。

01 定位启动阶段先判断控制权已经到达哪里

02 识别当前真实根区分 /、/sysroot 与 chroot 后的 /

03 区分状态寿命临时参数、持久条目、default target

04 选择最小操作只改变有证据支持的对象

05 完成分层验证配置、当前状态与功能分别证明

06 受控重启复验保证控制台、回退与同对象复查

## 专题地图 | 类型 | 专题 | |---|---| | 知识专题 | 从按下电源到 PID 1：启动链如何逐层交接 | | 操作专题 | 查询当前内核、启动条目与 GRUB environment | | 操作专题 | 临时启动参数与持久启动参数不能互相代替 | | 知识专题 | Target 的当前状态、默认状态与一次性目标 | | 知识专题 | rescue、emergency 与 `rd.break` 的环境边界 | | 操作专题 | 使用 `rd.break` 恢复 root 口令并保持 SELinux 正确 | | 诊断专题 | `fstab` 错误、启动证据与重启风险 | | 经典任务 | root 口令恢复与 `fstab` emergency 诊断 |

## 阅读时持续回答 1. 当前故障发生在 firmware、GRUB、initramfs 还是 systemd？ 2. 当前 `/` 是临时根还是真实根？ 3. 本次修改只影响一次启动，还是写入持久状态？ 4. `active target` 与 `default target` 分别回答什么？ 5. 当前证据能证明什么，又不能证明什么？ 6. 下一条最有区分度的证据是什么？ 7. 重启前是否有控制台、回退条目和明确验收项？

学习提示：遇到启动与恢复任务时，始终先确认启动阶段、真实根和状态寿命；任何修改都应由证据驱动，并为重启后的同对象复验预留控制台与回退路径。

## 第 30 章 • 正文

一台 Linux 主机从按下电源到出现登录提示符，并不是“系统启动了”这样一个不可再分的动作。控制权会在固件、引导加载程序、内核、initramfs、真实根文件系统和 systemd 之间逐层移交。每一层读取的配置不同，能产生的证据不同，能够安全执行的修复也不同。

恢复章节最危险的误判，通常不是不会敲命令，而是没有确认自己究竟处在哪一层：在 initramfs 的 `/` 中修改口令、把 GRUB 菜单中的一次性参数误认为持久配置、把 `set-default` 误认为当前已经切换，或在 `fstab` 尚未闭环时用重启反复试错。它们都属于“操作对象或状态寿命判断错误”。

本章按“启动对象 → 状态寿命 → 查询接口 → 恢复操作 → 证据链 → 重启验收”推进。systemd unit 的基础控制留给第 12 章；`fstab` 完整语法留给第 24 章；SELinux 模式和上下文模型留给第 28 章。本章只引入完成启动恢复所需的接口与边界。

### 概念

**Firmware** 是 BIOS 或 UEFI 等固件环境，负责基本硬件初始化、识别启动设备并进入下一阶段。它不理解 `/etc/fstab`、systemd unit 或 root 口令；如果尚未进入 kernel，systemd Journal 也不能为这一层提供证据。

### 概念

**Boot Loader / GRUB** 负责选择启动条目，加载 kernel 与 initramfs，并把 kernel command line 传给下一阶段。菜单中按 ``e`` 的编辑通常只影响一次启动；``grubby`` 管理的启动条目才承担持久参数，因此两者不能互相替代。

### 概念

**Kernel** 与 **initramfs** 分别承担内核初始化和早期用户空间。initramfs 包含发现真实根所需的驱动、工具与逻辑，把真实根挂载到 `/sysroot` 后再完成根切换；因此 initramfs 不是磁盘系统的普通 `/``。

### 概念

**rootfs** 与 `/sysroot` 是恢复操作中必须区分的路径语境。在 ``rd.break`` 中，当前 `/`` 是临时根，真实系统通常位于 `/sysroot`；只有重挂载正确对象并进入 ``chroot /sysroot``，`/etc/shadow``、`/etc/fstab`` 和 `/.autorelabel`` 才指向目标系统。

#### 概念

`systemd target` 是 `unit` 依赖集合与同步点，不是一个持续运行的“运行级别进程”。`active target` 描述当前已经到达的状态，`default target` 描述正常启动入口，`systemd.unit=` 则可以只覆盖一次启动。

#### 概念

`rescue`、`emergency` 与 `rd.break` 都能提供维护入口，但不在同一阶段。`rescue` 和 `emergency` 属于主系统的 `systemd` 目标；`rd.break` 在 `initramfs` 中暂停。环境不同，真实根位置、可用 `unit`、挂载状态和证据入口也不同。

#### 概念

`chroot` 改变当前进程及其子进程的路径解析根，使 `/etc`、`/var` 和 `/` 指向指定目录。它不会启动新 `kernel`、创建完整 `namespace` 或自动生成新的 `systemd`，因此“进入 `chroot`”只解决作用路径，不等于启动了一个独立系统。

#### 概念

`SELinux relabel` 是在口令恢复后重新建立文件安全上下文的持久恢复步骤。`touch /.autorelabel` 只是向下一次启动发出完整重标记请求；它不能证明重标记已经完成，也不能代替启动后对模式、失败 `unit` 和登录功能的检查。

# 操作语义速查

操作语义以下接口分别观察启动条目、GRUB 环境、target 状态、真实根、账号数据库、SELinux 重标记请求和启动证据。先确认作用对象，再决定参数。

## grubby

### SYNOPSIS

```
grubby --info=ALL
grubby --default-kernel
grubby --update-kernel=KERNEL --args="PARAMETERS"
grubby --update-kernel=KERNEL --remove-args="PARAMETERS"
```

查询或修改 RHEL 管理的 kernel 启动条目。它回答未来条目状态，不直接证明本次运行参数或重启成功。

#### --info=ALL

列出全部启动条目及关键字段。

#### --default-kernel

显示默认 kernel 路径。

#### --update-kernel=ALL

把后续参数操作作用于全部现有条目；使用前评估是否需要保留未修改回退条目。

#### --args=...

向目标条目持久添加参数。

#### --remove-args=...

从目标条目持久删除指定参数。

## grub2-editenv

### SYNOPSIS

```
grub2-editenv - list
grub2-editenv - unset menu_auto_hide
```

读取或修改默认 GRUB environment block。它不等同于 BLS 启动条目，也不等同于本次 kernel 的 `/proc/cmdline`。

#### - list

读取默认环境块中的变量。

#### - unset menu\_auto\_hide

取消自动隐藏菜单；这属于持久显示策略，不是一次 root 口令恢复的必要步骤。

## systemctl target 组

---

### SYNOPSIS

```
systemctl get-default
systemctl set-default TARGET
systemctl isolate TARGET
systemctl list-units --type=target --state=active
```

分开观察和修改 default target 与当前 active target。`set-default` 不立即切换，`isolate` 可能停止当前管理通道。

#### get-default

查询正常启动入口。

#### set-default TARGET

修改持久默认目标。

#### isolate TARGET

把当前系统切换到目标依赖集合，可能停止不再需要的 unit。

#### systemd.unit=TARGET

作为 kernel 参数，只覆盖一次启动目标。

## findmnt / mount

---

### SYNOPSIS

```
findmnt /
findmnt /sysroot
mount -o remount,rw /sysroot
```

先确认当前路径对应哪个挂载对象及其选项，再把真实根重新挂载为可写。不要凭提示符猜测 `/` 的语境。

#### findmnt PATH

显示目标、源、文件系统类型和挂载选项。

#### -o remount,rw

在不重新建立挂载的前提下，把指定挂载改为读写。

#### /sysroot

只在 initramfs 恢复语境中作为真实根的可能位置，仍应先查询确认。

## chroot / passwd

---

### SYNOPSIS

```
chroot /sysroot  
passwd root
```

把路径解析根切换到真实系统，再修改真实账号数据库。命令成功不等于作用对象正确，顺序不能颠倒。

```
chroot /sysroot
```

让后续绝对路径相对于真实根解析。

```
passwd root
```

交互式修改目标系统 root 口令；不要把明文口令写入命令历史。

## touch /.autorelabel

---

### SYNOPSIS

```
touch /.autorelabel
```

在真实根中创建重标记请求，让下一次正常启动重新建立 SELinux 文件上下文。它不是即时执行命令。

```
/.autorelabel
```

必须在 `chroot /sysroot` 之后创建，确保落在目标系统根目录。

启动边界

重标记可能耗时，不能强制中断；完成后仍检查 SELinux 模式、failed unit 和登录功能。

## journalctl boot 组

---

### SYNOPSIS

```
journalctl -b  
journalctl -k -b  
journalctl --list-boots  
journalctl -b -1
```

按启动轮次读取 systemd Journal。它适合 kernel、initramfs 后段和 systemd 阶段，但不能还原 firmware 尚未进入 kernel 的失败。

```
-b
```

限定当前启动。

```
-k -b
```



限定当前启动中的 kernel 消息。

`--list-boots`

列出当前可查询的启动轮次。

`-b -1`

读取上一次可用启动；无结果不等于上次没有事件。

## [知识专题] 从按下电源到 PID 1：启动链如何逐层交接

---

排查启动故障时，最先要回答的不是“用哪个修复命令”，而是“控制权已经走到哪里”。如果屏幕还停留在固件界面，`journalctl` 无能为力；如果 `rd.break` 已经中断在 `initramfs`，`/etc/shadow` 也不在当前 `/` 下；如果 `systemd` 已经进入 `emergency`，则 `failed unit` 和本次启动 `Journal` 才是高价值证据。启动阶段决定了证据入口和修复边界。

启动链纵向主模型

01

**Firmware** 初始化硬件并选择可启动入口

↓

02

**GRUB / Boot Loader** 选择条目，加载 kernel 与 `initramfs`，传递 `command line`

↓

03

**Kernel** 初始化内核能力并启动早期用户空间

↓

04

**initramfs** 发现存储，把真实根挂载到 `/sysroot`

↓

05

**switch\_root / rootfs** 从临时根切换到磁盘上的真实系统

↓

06

**systemd PID 1** 按 `default target` 及依赖启动 `unit`

↓

07

**Target** 与功能终态到达同步点后仍需验证挂载、服务和登录功能

### ① [知识点] firmware 选择启动设备，但不理解 Linux 的服务状态

主机接通电源后，BIOS 或 UEFI 完成基本初始化并按照固件配置选择可启动设备。传统 BIOS 与 UEFI 的磁盘布局和启动文件位置不同，但本章只需要掌握共同边界：firmware 的任务是找到并启动下一阶段，而不是读取 `/etc/fstab`、启动 `systemd` 服务或验证 `root` 密码。

固件阶段常见证据包括：

- 固件或 BMC 控制台上的硬件自检信息；
- 启动设备顺序；
- 是否能看到目标磁盘或 EFI 启动项；
- 是否已经出现 GRUB 菜单或 GRUB 错误。

如果目标磁盘根本没有被固件识别，继续修改 `grub.cfg` 不会解决问题。反过来，只要已经看到 GRUB 菜单，就说明控制权至少已离开 firmware 层。

## ② [知识点] GRUB 把启动条目解析为 kernel、initramfs 与 command line

GRUB 的核心输出不是“一个菜单”，而是一组可启动条目。一个条目至少关联：

- kernel image；
- initramfs image；
- kernel command line；
- 标题或条目标识；
- 可能的默认选择与回退信息。

RHEL 9 使用 Boot Loader Specification (BLS) 风格的启动条目，常见条目位于 `/boot/loader/entries/`。管理员通常通过 `grubby` 查询和修改条目，而不是直接手工改写生成后的 GRUB 主配置。GRUB environment 则保存类似菜单隐藏、已保存条目等引导环境变量，由 `grub2-editenv` 读取和修改。

GRUB 菜单中的按 `e` 编辑属于一次性启动编辑。它能临时追加 `rd.break`、`systemd.unit=` 或其他 kernel 参数，但正常情况下不会回写 BLS 条目。一次性编辑适合恢复和测试；需要长期生效时，必须改持久条目并再次查询确认。

## ③ [知识点] kernel 与 initramfs 不是同一个对象

boot loader 把 kernel 与 initramfs 放入内存并移交控制权。kernel 初始化内存管理、调度、设备模型等内核能力；initramfs 则提供一个临时的早期用户空间，包含启动所需的工具、脚本、systemd 组件和驱动模块。

initramfs 的价值在于解决“要挂载真实根，先得有访问真实根所需能力”的循环依赖。例如真实根位于 LVM、RAID、特定存储控制器或加密设备上，早期用户空间需要先装载相应模块、发现设备并建立映射，才能找到真正的 `/`。

`dracut` 用于生成 initramfs。它是重要帮助入口，但不能把“重建 initramfs”当作任何启动故障的默认答案。只有证据指向 initramfs 缺少模块、内容损坏或启动环境与硬件不匹配时，重建才是合理假设。

## ④ [知识点] initramfs 中的 `/` 与真实系统的 `/sysroot` 必须分开

在 initramfs 阶段，当前 `/` 是内存中的临时根。真实磁盘根通常被挂载到 `/sysroot`。这解释了 `rd.break` 口令恢复的两条关键命令：

```
mount -o remount,rw /sysroot
chroot /sysroot
```

第一条改变真实根挂载的读写属性；第二条改变当前进程的路径解析根。执行 `chroot /sysroot` 后，`/etc/shadow`、`/etc/fstab` 和 `/.autorelabel` 才指向目标系统中的文件。

如果在 `chroot` 前直接运行 `passwd root`，即使命令能够执行，它也可能操作 `initramfs` 环境中的文件，而不是磁盘系统的账号数据库。因此“命令成功”不能代替“作用对象正确”。

## ⑤ [知识点] 根切换后，磁盘系统中的 systemd 成为 PID 1

`initramfs` 完成真实根挂载后，启动流程通过根切换进入磁盘系统。之后 `systemd` 读取系统中的 `unit`、生成器输出和 `default target`，逐步启动 `mount`、`swap`、`service`、`socket` 等对象，最终达到文本登录、图形登录或指定维护目标。

这一阶段的高价值证据包括：

```
systemctl --failed
systemctl list-units --type=target --state=active
journalctl -b
journalctl -k -b
```

`systemctl --failed` 只列出 `systemd` 已知的失败 `unit`；`journalctl -b` 查看本次启动的记录；`journalctl -k -b` 聚焦本次启动的 `kernel` 消息。它们能帮助定位 `systemd`、`mount` 或 `kernel` 阶段的失败，却不能追溯尚未进入 `kernel` 的 `firmware` 故障。

### Cheatsheet

```
firmware
→ GRUB 选择条目并传递 command line
→ kernel + initramfs
→ 真实根挂载到 /sysroot
→ switch_root
→ systemd PID 1
→ default target
```

## [操作专题] 查询当前内核、启动条目与 GRUB environment

启动变更前必须先建立基线。查询时至少区分三种事实：当前正在运行什么、下次默认会启动什么、全部可用条目分别配置了什么。只看其中一个维度，很容易把当前运行状态误认为持久启动状态。

### ① [操作] 查询当前 kernel 与当前真正生效的 command line

作用对象：当前运行中的 `kernel`。

基本语义：`uname -r` 显示当前 kernel release；`/proc/cmdline` 显示本次启动时 kernel 实际收到的参数。

```
uname -r
cat /proc/cmdline
```

典型判断：

- `uname -r` 不能证明下一次默认仍会启动同一 kernel；
- `/proc/cmdline` 能证明本次参数，但不能证明持久条目已经更新；
- 临时在 GRUB 菜单追加的参数会出现在 `/proc/cmdline`，即使持久条目没有变化。

## ② [操作] 使用 `grubby` 查询默认 kernel 与全部启动条目

作用对象：RHEL 管理的 kernel 启动条目。

```
grubby --default-kernel
grubby --info=ALL
```

常见输出字段可能包括 `index`、`kernel`、`args`、`root`、`initrd`、`title` 和 `id`。审核时重点理解字段语义，而不要把某个小版本的字段顺序背成固定模板。

`index` 可能随着 kernel 安装、删除或条目变化而改变。需要长期引用某个条目时，kernel 路径、条目 ID 或明确标题通常比记住裸索引更稳妥。

## ③ [操作] 使用 `grub2-editenv` 读取 GRUB environment

作用对象：GRUB environment block 中的变量。

```
grub2-editenv - list
```

这个命令可用于查看保存的条目、菜单隐藏等环境变量。它不等同于读取全部 BLS 启动条目，也不等同于查看当前 kernel command line。三者分别回答：

```
GRUB environment: 引导环境变量是什么
BLS/grubby: 启动条目是什么
/proc/cmdline: 本次 kernel 实际收到什么
```

## ④ [操作] 在菜单可能隐藏时进入 GRUB

RHEL 9 的某些版本和安装状态可能默认隐藏 GRUB 菜单。启动时可尝试按 `Esc`、`F8` 或按住 `Shift` 进入菜单，具体按键时机受固件、控制台和虚拟化环境影响。

如果业务需求是让菜单长期显示，可以调查 `menu_auto_hide` 环境变量；但为了执行一次 `rd.break` 恢复，没有必要先永久改变菜单显示策略。恢复操作应尽量减少持久变更。

## ⑤ [操作] 建立变更前基线

在修改 kernel 参数或默认条目前，推荐保存以下证据：

```
uname -r
cat /proc/cmdline
grubby --default-kernel
grubby --info=ALL
grub2-editenv - list
```

工作环境中还应把输出保存到变更记录。考试环境中至少要在头脑中明确：当前 kernel、默认 kernel、目标条目和需要修改的参数分别是什么。

验证与边界：查询命令本身不会证明目标 kernel 可以成功启动。持久条目修改后的最终验证仍需要受控重启，但应先完成静态核对和风险评估。

### Cheatsheet

```
当前 kernel: uname -r
当前参数: cat /proc/cmdline
默认 kernel: grubby --default-kernel
全部条目: grubby --info=ALL
GRUB 环境: grub2-editenv - list
```

## [操作专题] 临时启动参数与持久启动参数不能互相代替

同一个 kernel 参数可以通过 GRUB 菜单临时追加，也可以通过 `grubby` 写入启动条目。两种方式看起来都能让参数“生效”，但它们的生命周期完全不同。恢复时偏向一次性参数；长期配置则必须修改持久条目，并在变更前后分别查询。

### ① [操作] 在 GRUB 菜单中临时编辑一次启动

作用对象：当前选中的一次启动过程。

典型流程：

1. 进入 GRUB 菜单；
2. 选择要启动的条目；
3. 按 `e` 编辑；
4. 定位包含 kernel 路径和现有参数的行；
5. 在行末追加所需参数；
6. 按界面提示继续启动，常见为 `Ctrl+x`。

例如进入 `initramfs` 断点：

```
rd.break
```

或一次性进入 rescue:

```
systemd.unit=rescue.target
```

只追加需要的参数，不把“删除其他参数直到只剩 `ro`”写成通用步骤。无证据删除 `root=`、LVM、console、crashkernel 或其他参数，可能制造新的启动故障。

## ② [操作] 用 `grubby` 持久添加 kernel 参数

作用对象：指定 kernel 条目或全部 kernel 条目。

```
grubby --update-kernel=ALL --args="<PARAMETERS>"
```

对单一 kernel 可用明确的 kernel 路径替换 `ALL`。选择范围前先回答：

- 参数是否应作用于所有已安装 kernel；
- 是否只为当前默认 kernel 添加；
- 是否需要保留一个未修改的已知可启动条目作为回退。

## ③ [操作] 用 `grubby` 删除持久 kernel 参数

```
grubby --update-kernel=ALL --remove-args="<PARAMETERS>"
```

删除时要传入需要移除的参数，而不是通过手工编辑大段启动行来“清理”。参数可能包含值，应以查询到的实际形式为准。

## ④ [操作] 分别验证未来条目与当前运行状态

修改后先查询条目：

```
grubby --info=ALL
```

在早期 RHEL 9.0 环境或完成 kernel 安装、删除之后，更应重新检查所有条目，不要假定新条目一定继承了预期参数。这里的安全原则不是记住某个版本特例，而是把“安装结果”重新还原成可查询的启动条目。

重启前，这只能证明条目文本发生变化。完成受控重启后，再检查：

```
cat /proc/cmdline  
uname -r
```

二者形成两个验证层：

grubby 输出：未来启动条目  
/proc/cmdline：本次实际生效参数

### ⑤ [边界] 不直接把生成后的 grub.cfg 当作首选配置真源

RHEL 9 的启动条目、BLS 和 GRUB 主配置存在生成关系。日常 kernel 参数管理优先使用 grubby。直接编辑生成后的 grub.cfg 容易被后续更新覆盖，也更容易误改复杂逻辑。

本章不完整展开 GRUB 重装、手工恢复 core image、Secure Boot 密钥或复杂串口菜单配置。只有启动条目和参数管理属于本章主线。

#### Cheatsheet

一次启动：GRUB 菜单按 e，追加参数  
持久添加：grubby --update-kernel=... --args="..."  
持久删除：grubby --update-kernel=... --remove-args="..."  
条目验证：grubby --info=ALL  
生效验证：cat /proc/cmdline

## [知识专题] Target 的当前状态、默认状态与一次性启动目标

target 常被简化成传统“运行级别”的替代物，但真正有用的理解是：target 是一组 unit 依赖和同步关系。系统可以同时有多个 active target；default target 只是正常启动时的入口；isolate 改变当前状态；systemd.unit= 则只覆盖一次启动。把这些状态混在一起，会出现“已经 set-default，为什么现在还没切换”或“已经 isolate，为什么重启又回去了”的误判。

### ① [知识点] 常见 target 代表不同启动终态

| Target            | 主要含义               | 典型用途        |
|-------------------|--------------------|-------------|
| multi-user.target | 多用户、文本环境及其依赖       | 常见服务器默认目标   |
| graphical.target  | 在多用户基础上加入图形登录能力    | 桌面或图形管理环境   |
| rescue.target     | 单用户维护环境，加载较多基本系统能力 | 维护真实系统      |
| emergency.target  | 极小维护环境，只拉起非常少的依赖   | 处理严重启动或挂载故障 |

graphical.target 通常依赖 multi-user.target，因此系统可能同时显示多个 active target。不能把 list-units --type=target 的多行结果解释为“系统同时处于多个传统运行级别”。

## ② [操作] 查询和设置 default target

```
systemctl get-default  
systemctl set-default multi-user.target
```

`get-default` 回答“正常启动默认从哪个 target 开始”；`set-default` 改变持久默认值，但不会自动停止当前图形会话或立即切换当前系统。

修改后再次执行：

```
systemctl get-default
```

这只能验证持久配置，不证明系统已经在新 target 的运行状态中。

## ③ [操作] 查询当前 active target

```
systemctl list-units --type=target --state=active
```

当前 active target 是运行状态证据。若要聚焦某个目标，可查询：

```
systemctl is-active multi-user.target  
systemctl status rescue.target
```

`active` 只说明该 target 已达到，不自动证明该目标下的每个业务功能都健康。例如到达 `multi-user.target` 不等于自定义 Web 应用一定可用。

## ④ [操作] 使用 `isolate` 切换当前目标

```
systemctl isolate rescue.target
```

`isolate` 会启动目标所需 unit，并停止不属于目标依赖的其他 unit。它可能中断：

- 图形会话；
- SSH 或网络连接；
- 业务服务；
- 依赖被停止服务的监控或管理通道。

因此远程主机上执行前必须确认控制台或回退通道。考试环境也应理解该操作的当前影响，不把它当作无风险的“查看模式”。

## ⑤ [操作] 用 kernel 参数指定一次性启动 target

在 GRUB kernel command line 追加：



```
systemd.unit=rescue.target
```

或：

```
systemd.unit=emergency.target
```

该参数只影响本次启动，不改变 `systemctl get-default` 的持久默认目标。维护完成后，若当前 `systemd` 环境允许，可以尝试：

```
systemctl default
```

让系统回到 `default target`；也可以在风险评估后正常重启。

## ⑥ [比较] 当前、默认与一次性的三维状态

```
当前 active: systemctl list-units / is-active  
持久 default: systemctl get-default / set-default  
一次启动: systemd.unit=<target>
```

这三个维度必须分别验证。任何一个维度都不能替代另外两个。

### Cheatsheet

```
默认: get-default / set-default  
当前: list-units --type=target / isolate  
一次启动: systemd.unit=
```

## [知识专题] rescue、emergency 与 `rd.break` 的环境边界

三种入口都可能出现 `root shell`，因此最容易被混为同一模式。区分它们的关键不是提示符长什么样，而是 `systemd` 已经执行到哪里、真实根挂载在哪里、哪些依赖已经启动、修改哪个路径才会影响磁盘系统。

### ① [知识点] `rescue.target` 已经进入真实系统

`rescue target` 通常在真实根上运行，提供较完整的基本系统与单用户维护环境。它适合：

- 在真实系统中修改配置；
- 停止大部分业务后维护；
- 处理不需要早期用户空间介入的问题；
- 从当前系统切换到较小运行集合。

进入后仍应确认根挂载状态和必要文件系统，不要仅凭“rescue”名称假设所有挂载都可写。

## ② [知识点] `emergency.target` 只拉起极少依赖

`emergency.target` 比 `rescue` 更小，常用于 `mount`、`local-fs` 或关键依赖失败后的维护。很多教材把它简写为“根只读”，但实际挂载状态与进入路径、生成的 `mount unit` 和系统版本有关。正确做法是查询：

```
findmnt /  
findmnt -no OPTIONS /
```

需要写入时再根据证据执行：

```
mount -o remount,rw /
```

## ③ [知识点] `rd.break` 在 `initramfs` 阶段中断

`rd.break` 是 `dracut` 早期用户空间的断点参数。它在根切换前提供 `shell`，常用于 `root` 口令恢复。此时：

- 当前 `/` 属于 `initramfs`；
- 真实系统通常位于 `/sysroot`；
- `/sysroot` 可能以只读方式挂载；
- 正常系统的 `systemd` 尚未完整启动。

因此 `systemctl get-default`、普通服务状态和磁盘系统路径不能按正常启动环境理解。

## ④ [操作] 根据环境选择正确的重挂载对象

在正常系统、`rescue` 或 `systemd emergency` 中，目标通常是当前根：

```
findmnt /  
mount -o remount,rw /
```

在 `rd.break` 的 `initramfs shell` 中，目标通常是真实根：

```
findmnt /sysroot  
mount -o remount,rw /sysroot
```

操作前先查询，不把命令顺序背成脱离环境的口诀。

## ⑤ [知识点] `chroot` 只改变路径解析根，不创建完整新系统

```
chroot /sysroot
```

执行后，当前进程将 `/sysroot` 视作 `/`。它适合让 `passwd`、编辑器和路径操作作用于真实系统。但 `chroot`：

- 不启动新的 kernel；
- 不自动创建 PID、网络或 mount namespace；
- 不保证 `/proc`、`/sys`、`/dev` 都具备完整绑定；
- 不等于容器或虚拟机；
- 不会自动让磁盘系统的 `systemd` 成为当前 PID 1。

`root` 口令恢复只需要有限路径和命令，因此课程流程通常可以直接 `chroot /sysroot`。更复杂的外部救援环境可能需要额外挂载伪文件系统，但不属于本章经典任务主线。

⑥ [比较] 三种环境的判断表

| 入口        | 阶段            | 真实根常见位置  | 主要证据                               | 典型用途        |
|-----------|---------------|----------|------------------------------------|-------------|
| rescue    | 正常<br>systemd | /        | systemctl、Journal、findmnt /        | 单用户维护       |
| emergency | 极小<br>systemd | /        | failed unit、Journal、findmnt /      | 严重依赖或挂载故障   |
| rd.break  | initramfs     | /sysroot | /proc/cmdline、<br>findmnt /sysroot | 口令恢复、早期启动调试 |

Cheatsheet

rescue/emergency: 真实根通常是 /  
rd.break: 真实根通常是 /sysroot  
任何环境: 先 findmnt, 再 remount

[操作专题] 使用 `rd.break` 恢复 root 口令并保持 SELinux 正确

`root` 口令恢复是本章最经典的考试任务，但真正的评分对象不只是 `passwd` 成功。完整终态包括：修改发生在真实根、口令数据库可持久使用、SELinux 标签在下一次启动后正确、临时参数没有写入持久条目、系统能够正常进入目标状态。

① [操作] 在 GRUB 中只为本次启动追加 `rd.break`

1. 重启并进入 GRUB 菜单；
2. 选择目标 kernel 条目；
3. 按 `e` 编辑；
4. 定位 kernel command line；

5. 在现有参数末尾追加：

```
rd.break
```

6. 按界面提示继续启动，常见为 `Ctrl+x`。

不要无调查删除现有 `root=`、LVM、console 或其他参数。只添加断点参数即可达到恢复目的。

## ② [操作] 证明 `/sysroot` 是目标真实根并重挂载为读写

进入 shell 后先查询：

```
cat /proc/cmdline  
findmnt /sysroot
```

确认 command line 包含 `rd.break`，并识别 `/sysroot` 的来源和挂载选项。然后执行：

```
mount -o remount,rw /sysroot  
findmnt -no TARGET,SOURCE,FSTYPE,OPTIONS /sysroot
```

第二次 `findmnt` 用于验证 `rw` 已生效。不能把 `mount` 无报错扩大为“真实根一定可写”。

## ③ [操作] `chroot` 到真实系统后修改口令

```
chroot /sysroot  
passwd root
```

推荐交互输入题目给出的新口令。这样避免把明文口令保存在命令行、Shell 历史或进程参数中。考试模拟答案中常见的管道或 `--stdin` 写法虽然可能可用，但不是本讲义的默认安全答案。

完成后可检查命令退出状态，但不要尝试读取或展示 `/etc/shadow` 中的口令哈希作为“验证”。正确验证应在正常启动后进行认证。

## ④ [操作] 在真实根中创建 SELinux 重标记请求

仍处于 `chroot` 时执行：

```
touch /.autorelabel  
ls -l /.autorelabel
```

修改口令会改写安全相关文件。在 SELinux Enforcing 系统上，从 `initramfs chroot` 修改文件后，应让下一次启动执行重标记。`touch /.autorelabel` 创建的是下一次启动请求，不是立即完成重标记。

如果在 chroot 前执行 `touch /.autorelabel`，文件会创建在 initramfs 的 `/` 中，退出后不会留在真实系统。因此路径语境是该步骤的核心。

⑤ [操作] 依次退出 chroot 与 initramfs shell

```
exit
exit
```

第一次退出 chroot，回到 initramfs shell；第二次退出断点 shell，让启动继续。也可以使用对应快捷键发送 EOF，但教学和答题时写出两次 `exit` 更能明确环境层次。

不要在重标记过程中强制断电。文件较多或存储较慢时，首次启动可能明显变长。

⑥ [验证] 正常启动后的分层验收

启动完成后至少检查：

```
cat /proc/cmdline
getenforce
systemctl --failed
journalctl -b -p err
```

验收含义：

- `/proc/cmdline` 不应因为本次临时编辑而把 `rd.break` 变成持久参数；
- `getenforce` 应符合原系统预期，不能以关闭 SELinux 代替恢复；
- `systemctl --failed` 用于发现本次启动的 unit 失败；
- Journal 用于确认重标记或启动过程中的错误；
- 最终还要用新 root 口令完成实际认证。

⑦ [典型错误] 命令顺序看似相似，作用对象却不同

| 错误                                           | 为什么错误             | 正确修正                                        |
|----------------------------------------------|-------------------|---------------------------------------------|
| 在 chroot 前执行 <code>passwd root</code>        | 可能修改 initramfs 环境 | 先 <code>chroot /sysroot</code>              |
| 未重挂载 <code>/sysroot</code> 为 <code>rw</code> | 口令或标记文件无法持久写入     | <code>findmnt</code> 后 <code>remount</code> |
| 在 chroot 前 <code>touch /.autorelabel</code>  | 标记文件创建在临时根        | 在 chroot 内创建                                |
| 通过关闭 SELinux 避免重标记                           | 改变安全终态并掩盖标签问题     | 保持原模式并重标记                                   |
| 把 <code>rd.break</code> 写入全部条目               | 恢复参数永久生效          | 只在 GRUB 菜单临时追加                              |
| 重标记时强制重启                                     | 可能留下不完整标签状态       | 等待流程完成                                      |

```
GRUB 追加 rd.break
→ findmnt /sysroot
→ mount -o remount,rw /sysroot
→ chroot /sysroot
→ passwd root
→ touch /.autorelabel
→ exit
→ exit
→ 正常启动后分层验证
```

## [诊断专题] `/etc/fstab` 错误为什么会把系统带入 emergency

systemd 会根据 `/etc/fstab` 生成 mount unit，并把关键本地文件系统纳入启动依赖。如果设备标识、文件系统类型、选项或目标不正确，mount unit 可能超时或失败，进而阻止 `local-fs.target` 达成，系统可能进入 emergency。正确修复不是“看到 emergency 就注释 fstab”，而是把 failed unit、日志、设备身份和具体配置行对应起来。

### ① [症状] emergency 是结果，不是根因

常见表征包括：

- 控制台提示进入 emergency mode；
- 某个 `.mount` unit 失败；
- 启动长时间等待设备；
- `local-fs.target` 或相关依赖未能完成；
- 提示检查 Journal 或输入 root 口令维护。

同样的表征可能来自不同原因：UUID 不存在、设备尚未出现、文件系统类型错误、选项拼写错误、挂载点冲突或远程文件系统依赖不满足。不要用单一修复猜全部故障。

### ② [当前证据] 先读取 failed unit 与本次启动日志

```
systemctl --failed
journalctl -b -p err
journalctl -b
```

如果已知失败 unit，可聚焦：

```
systemctl status <NAME.mount>
journalctl -b -u <NAME.mount>
```

目标是得到：

- 失败的 unit 名称；
- 对应挂载点；
- systemd 或 mount 报告的具体错误；
- 是否为设备超时、类型不符或选项错误。

### ③ [假设] 将 `.mount` unit 映射回挂载点和 `fstab` 行

systemd 的 mount unit 名称由挂载路径转义得到。例如 `/data` 常对应 `data.mount`，更深路径会被编码。可以使用：

```
systemd-escape --path --suffix=mount /data
```

然后检查：

```
grep -nEv '^s*(#|$)' /etc/fstab  
findmnt --fstab
```

不要一次删除或注释多行。先锁定失败 unit 对应的唯一记录。

### ④ [下一条证据] 比较配置声明与真实设备身份

```
lsblk -f  
blkid  
findmnt --verify
```

重点比较：

- `UUID=` 或 `LABEL=` 是否真实存在；
- `FSTYPE` 与 `fstab` 第三字段是否一致；
- 目标目录是否合理；
- 选项是否被当前文件系统支持；
- 设备是否本应在启动时存在；
- 远程文件系统是否需要网络依赖或自动挂载策略。

`findmnt --verify` 是静态检查入口，但它不能代替真实挂载。静态通过后仍需执行受控挂载验证。

### ⑤ [最小修复] 只修正有证据支持的字段

典型最小修复包括：

- 用 `blkid` 看到的真实 UUID 替换错误 UUID；
- 修正文件系统类型；

- 删除确认无效的选项；
- 修正目标目录；
- 对确属非关键、允许缺失的设备评估 `nofail`，但不能为了通过启动而掩盖必需挂载失败。

本章不教授格式化、重建文件系统或高风险修复工具。已有数据设备出现错误时，不执行 `mkfs`、`wipefs -a` 或未经调查的强制修复。

## ⑥ [再验证] 重新加载、模拟挂载并检查终态

修改 `/etc/fstab` 后：

```
systemctl daemon-reload
findmnt --verify
mount -a
findmnt <TARGET>
systemctl --failed
```

这些证据分别回答：

- `systemd` 是否重新读取生成关系；
- 静态语法和引用是否合理；
- 所有可尝试的 `fstab` 挂载是否执行；
- 目标路径实际挂载到什么源；
- 是否仍有失败 `unit`。

`mount -a` 无输出或退出成功仍不能证明目标挂载正确。必须用 `findmnt <TARGET>` 检查 `source`、`fstype` 和 `options`。

## ⑦ [重启验证] 当前闭环后再进行受控重启

只有在当前环境中完成以下条件后，才进入重启验证：

```
失败 unit 已定位
→ 配置只做最小修复
→ findmnt --verify 通过目标检查
→ mount -a 没有目标错误
→ findmnt 显示正确 source
→ systemctl --failed 无相关失败
```

重启后再次执行：

```
systemctl --failed
findmnt <TARGET>
journalctl -b -p err
```

这才证明持久启动路径正确。



```
emergency 症状
→ systemctl --failed
→ journalctl -b
→ 映射 mount unit 与 fstab
→ lsblk -f / blkid
→ findmnt --verify
→ 最小修复
→ daemon-reload + mount -a
→ findmnt 目标
→ 受控重启复验
```

## [诊断专题] 按启动阶段选择证据：控制台、kernel log 与 boot Journal

“查日志”不是一条万能建议。日志系统本身也要经过启动过程才能工作。证据选择应跟随阶段：firmware 与 GRUB 主要依赖控制台；kernel 与 initramfs 依赖控制台和 kernel 消息；systemd 阶段才适合通过 boot Journal、failed unit 和 target 状态推进。

### ① [诊断] firmware 与 GRUB 前后的证据边界

如果屏幕停留在：

- BMC 或固件自检；
- 无可启动设备；
- EFI 启动项错误；
- GRUB 自身错误或菜单无法加载；

则 systemd Journal 尚未形成。优先保存控制台信息、固件启动顺序和 GRUB 表征。不要因为 `journalctl -b -1` 没有相关记录，就断言 firmware 没有失败。

### ② [诊断] kernel 与 initramfs 阶段

若已经看到 kernel 或 initramfs 错误，可关注：

- 当前 kernel command line；
- 根设备、LVM 或驱动发现信息；
- initramfs emergency shell；
- 控制台上的 kernel 报错。

如果系统后来成功进入用户空间，可以查询本次 kernel 记录：

```
journalctl -k -b
```

也可按关键字检索，但不要只靠 `grep` 一条错误就停止调查。需要结合前后文和时间顺序。

### ③ [诊断] systemd 阶段的三条主证据

```
systemctl --failed
journalctl -b
journalctl -b -p err
```

它们分别提供：

- `systemd` 当前标记为失败的 `unit`；
- 本次启动完整 `Journal`；
- 本次启动较高优先级的错误记录。

优先级筛选能减少噪声，但某些关键因果记录可能是 `notice`、`warning` 或普通信息，因此不能只保留 `error` 级别。

### ④ [诊断] 区分本次与前次启动

```
journalctl --list-boots
journalctl -b 0
journalctl -b -1
```

`-b 0` 是本次启动，`-b -1` 是上一次可用启动记录。前次记录能否查看，取决于 `Journal` 是否持久保存以及记录是否被轮转。没有前次记录不自动意味着系统从未失败。

### ⑤ [验证边界] 日志、target 与功能终态不是同一个证明

- `Journal` 中出现“`Started`”通常只说明 `unit` 启动动作成功；
- `target active` 只说明依赖同步点达到；
- `systemctl --failed` 为空不代表应用功能和数据都正确；
- 某条 `error` 也不一定是当前故障的根因。

启动诊断最终仍要回到目标：能否登录、目标文件系统是否正确挂载、服务是否提供功能、系统是否能在无需人工干预时启动。

#### Cheatsheet

```
firmware/GRUB: 控制台证据
kernel/initramfs: 控制台 + /proc/cmdline + kernel log
systemd: failed unit + journalctl -b
上次启动: --list-boots + -b -1
```

## [诊断专题] 重启风险、回退路径与最小变更

启动相关变更的特殊风险在于：当前系统可能运行正常，而错误只有在下一次引导时才暴露。如果唯一的远程通道依赖当前网络和服务，错误重启可能直接失去管理能力。因此“配置完成，重启看看”不是稳健方法。

### ① [操作] 变更前记录可回退基线

```
uname -r
cat /proc/cmdline
grubby --default-kernel
grubby --info=ALL
systemctl get-default
systemctl --failed
findmnt --verify
```

对 `fstab` 或启动条目做工作变更时，还应备份原文件或保存差异。考试环境可以简化记录形式，但不能省略对象识别。

### ② [边界] 一次只改变一个启动维度

高风险组合包括同时：

- 更换默认 kernel；
- 修改全部 kernel 参数；
- 改 default target；
- 改多条 `fstab`；
- 重建 `initramfs`；
- 删除旧启动条目。

多个变量同时改变后，即使启动失败，也难以定位是哪一步造成。优先最小变更、逐层验证，并保留已知可启动条目。

### ③ [边界] `systemctl isolate` 可能切断当前管理通道

`isolate rescue.target` 或 `isolate emergency.target` 会停止目标不需要的 unit。远程执行前要确认：

- 是否有控制台或 BMC；
- 网络和 SSH 是否会被停止；
- 业务能否接受服务中断；
- 当前 Shell 是否属于即将被停止的会话。

考试任务若只要求设置下次默认目标，不应为了“验证”无必要地 `isolate` 当前系统。

#### ④ [边界] SELinux 重标记不是普通快速重启

创建 `/.autorelabel` 后，下一次启动会遍历文件系统并恢复标签。系统规模越大，耗时越长。期间强制断电可能留下不完整状态。正确操作是：

- 保持控制台可见；
- 等待重标记和后续重启完成；
- 启动后检查 SELinux 模式、失败 unit 和 Journal；
- 不用永久关闭 SELinux 规避等待。

#### ⑤ [边界] 不使用破坏性捷径掩盖启动故障

本章禁止把以下动作作为默认答案：

- 删除大段现有 kernel 参数；
- 关闭 SELinux；
- 对必需挂载随意加入 `nofail`；
- 未调查就重建 GRUB 或 initramfs；
- 对已有数据设备执行 `mkfs`、`wipefs -a`；
- 在重标记或文件系统写入中强制断电；
- 把“命令没有报错”当作重启后终态。

#### ⑥ [验证] 重启前后使用同一组对象重新查询

重启前：

```
启动条目、default target、fstab、failed unit、目标挂载
```

重启后：

```
uname -r、/proc/cmdline、active target、目标挂载、failed unit、boot Journal
```

使用同一对象的前后证据，才能证明持久变更真正生效，而不是只在当前会话中暂时可用。

#### Cheatsheet

先基线

- 最小变更
- 静态与当前验证
- 确认控制台/回退
- 受控重启
- 用同一对象复验

# [经典任务] 使用 `rd.break` 恢复 root 口令

## 任务环境

一台启用了 SELinux Enforcing 的 RHEL 9 主机可以通过本地控制台访问。系统能够到达 GRUB，但 root 口令未知。题目要求把 root 口令修改为考场给定的新值。现有 default target、kernel 选择和其他 kernel 参数均不得永久改变。

## 当前状态

- 可以进入 GRUB 菜单并编辑目标启动条目；
- root 口令未知，无法以 root 正常认证；
- 系统使用 SELinux Enforcing；
- 当前没有理由关闭 SELinux 或重建 GRUB；
- 本会话没有可执行 live test 的 RHEL 9 VM。

## 目标终态

1. 真实磁盘系统的 root 口令已更新；
2. SELinux 保持原预期模式；
3. 下一次启动完成必要的文件标签重标记；
4. `rd.break` 只用于本次启动，不出现在持久条目；
5. default target 不变；
6. 系统能够正常启动并接受新 root 口令认证；
7. 无与本次恢复相关的失败 unit。

## 限制条件

- 不使用 `selinux=0` 或 `enforcing=0`；
- 不删除原 kernel command line 中的其他参数；
- 不把新口令明文写入命令历史或答案日志；
- 不把 `rd.break` 写入持久启动条目；
- 不在 SELinux 重标记过程中强制断电；
- 不以读取 `/etc/shadow` 哈希作为口令验证。

## 验收矩阵

| 层次 | 验收对象         | 推荐证据                           | 能证明什么                        |
|----|--------------|--------------------------------|------------------------------|
| 阶段 | initramfs 断点 | <code>cat /proc/cmdline</code> | 本次包含 <code>rd.break</code>   |
| 根  | 真实根          | <code>findmnt /sysroot</code>  | <code>/sysroot</code> 的来源与选项 |

| 层次      | 验收对象     | 推荐证据                                                         | 能证明什么                       |
|---------|----------|--------------------------------------------------------------|-----------------------------|
| 写入      | 根可写      | <code>findmnt -no OPTIONS /sysroot</code>                    | 已包含 <code>rw</code>         |
| 作用域     | chroot   | 当前路径语境                                                       | <code>passwd</code> 作用于真实系统 |
| SELinux | 重标记请求    | <code>ls -l /.autorelabel</code>                             | 文件创建在真实根                    |
| 持久性     | 临时参数     | 正常启动后 <code>/proc/cmdline</code>                             | <code>rd.break</code> 未持久化  |
| 安全模式    | SELinux  | <code>getenforce</code>                                      | 模式保持预期                      |
| 系统状态    | unit 与日志 | <code>systemctl --failed</code> 、 <code>journalctl -b</code> | 启动未留下相关失败                   |
| 功能      | root 认证  | 使用新口令登录                                                      | 目标口令真正可用                    |

## [参考解答] 使用 `rd.break` 恢复 root 口令

### 一、调查与进入正确阶段

- 通过本地控制台重启主机并进入 GRUB 菜单。
- 选择需要启动的 kernel 条目，按 `e` 编辑。
- 定位 kernel command line，在现有参数末尾仅追加：

```
rd.break
```

- 按界面提示继续启动，常见为 `Ctrl+x`。

判断边界：本步骤只需要增加断点，不删除其他参数，也不调用 `grubby`。使用 `grubby` 会把恢复参数写入持久条目，改变任务边界。

### 二、确认真实根并改为读写

进入 `initramfs shell` 后：

```
cat /proc/cmdline
findmnt /sysroot
```

确认本次参数包含 `rd.break`，并确认 `/sysroot` 是真实系统根。然后：

```
mount -o remount,rw /sysroot
findmnt -no TARGET,SOURCE,FSTYPE,OPTIONS /sysroot
```

只有看到 `rw` 后，才进入下一步。

### 三、进入真实系统并修改口令

```
chroot /sysroot  
passwd root
```

根据交互提示输入题目给定的新口令。不要在答案中写出真实口令值，也不要使用包含明文口令的管道作为默认流程。

### 四、请求 SELinux 重标记

仍在 chroot 中执行：

```
touch /.autorelabel  
ls -l /.autorelabel
```

这里的 `/` 已经是真实根。若先退出 chroot 再创建文件，标记请求会落在 initramfs 临时根中，无法影响正常系统。

### 五、退出两层环境并等待启动

```
exit  
exit
```

第一次退出 chroot，第二次退出 initramfs 断点。系统继续启动并执行 SELinux 重标记。等待流程完成，不强制断电。

### 六、正常启动后的分层验证

```
cat /proc/cmdline  
getenforce  
systemctl get-default  
systemctl --failed  
journalctl -b -p err
```

然后使用新 root 口令完成实际认证。

验证解释：

- `/proc/cmdline` 不含持久 `rd.break`；
- `getenforce` 保持原安全终态；
- default target 未被改变；
- failed unit 与 Journal 没有与恢复相关的未解决错误；
- 实际认证证明口令终态，而不是只证明 `passwd` 命令返回成功。

## 七、典型错误与修复

- 错误：在 chroot 前运行 `passwd`。修复：重新确认 `/sysroot`，进入 chroot 后再改真实账号数据库。
- 错误：未把 `/sysroot` remount 为 `rw`。修复：用 `findmnt` 验证选项，不凭命令无输出猜测。
- 错误：在 `initramfs` 根创建 `/.autorelabel`。修复：在 chroot 内创建。
- 错误：用关闭 SELinux 规避标签问题。修复：保持原模式并完成重标记。
- 错误：重标记期间强制重启。修复：等待完成，启动后再检查 Journal。

## [经典任务] 诊断错误 `fstab` 导致的 emergency

---

### 任务环境

一台 RHEL 9 主机在修改 `/etc/fstab` 后进入 emergency。目标数据文件系统仍存在且包含数据，不允许格式化或重新创建。emergency shell 可用，题目要求恢复无人干预启动，并确保目标文件系统仍按原计划持久挂载。

### 当前状态

- 至少一个 `.mount` unit 失败；
- `/etc/fstab` 中有一条设备标识、文件系统类型或选项与真实设备不一致；
- 真实 UUID、LABEL 和文件系统类型必须从本机证据获得；
- 目标挂载属于必需挂载，不能通过注释或随意增加 `nofail` 隐藏问题。

### 目标终态

1. 找到实际失败的 `mount` unit；
2. 将失败 unit 映射到唯一的 `fstab` 行；
3. 只修正有证据支持的字段；
4. `findmnt --verify` 不再报告目标错误；
5. `mount -a` 能处理目标条目；
6. `findmnt <TARGET>` 显示正确 source、fstype 和 options；
7. 相关 failed unit 消失；
8. 系统能进入原 default target；
9. 受控重启后仍能无人干预启动并挂载目标。

### 限制条件

- 不执行 `mkfs`、`wipefs -a` 或重新创建已有文件系统；
- 不注释必需挂载；
- 不用 `nofail` 掩盖故障；
- 不在定位前反复 reboot；



- 不把 `mount -a` 无输出当作唯一验收；
- 不编造 UUID、设备名或真实输出。

验收矩阵

| 层次      | 推荐证据                                          | 目标判断            |
|---------|-----------------------------------------------|-----------------|
| 症状      | <code>systemctl --failed</code>               | 找到失败 mount unit |
| 原因      | <code>journalctl -b -u &lt;unit&gt;</code>    | 得到具体错误类型        |
| 配置      | <code>grep -nEv '^s*(# \$)' /etc/fstab</code> | 定位对应行           |
| 设备      | <code>lsblk -f</code> 、 <code>blkid</code>    | 获取真实身份和类型       |
| 静态      | <code>findmnt --verify</code>                 | 配置引用和语法合理       |
| 当前      | <code>mount -a</code>                         | 可执行挂载动作         |
| 终态      | <code>findmnt &lt;TARGET&gt;</code>           | 目标实际挂载正确        |
| systemd | <code>systemctl --failed</code>               | 相关失败已消失         |
| 持久      | 重启后复验                                         | 启动链和挂载持久正确      |

[参考解答] 诊断错误 `fstab` 导致的 emergency

一、从症状取得当前证据

进入 emergency shell 后，先确认根是否可写：

```
findmnt /
findmnt -no OPTIONS /
```

如果需要修改而根为只读：

```
mount -o remount,rw /
findmnt -no OPTIONS /
```

随后查询失败对象：

```
systemctl --failed
journalctl -b -p err
```

若已经看到失败的 mount unit，例如 `<NAME.mount>`：

```
systemctl status <NAME.mount>
journalctl -b -u <NAME.mount>
```

## 二、把 unit 映射到具体配置行

根据挂载点推导 unit 名称，或反向查看 unit 的 Where= / 状态信息。需要时可使用：

```
systemd-escape --path --suffix=mount <TARGET>
```

检查有效 `fstab` 行：

```
grep -nEv '^\s*(#|$)' /etc/fstab
findmnt --fstab
```

只锁定目标行，不批量注释其他记录。

## 三、用设备证据验证假设

```
lsblk -f
blkid
findmnt --verify
```

比较：

- `fstab` 的 UUID/LABEL 是否存在；
- 文件系统类型是否一致；
- 挂载点是否正确；
- 选项是否有效；
- 设备是否应在本地启动阶段出现。

根据实际证据修改 `/etc/fstab`。答案中不填写虚构 UUID；实际考试应复制本机 `blkid` 或 `lsblk -f` 给出的值。

## 四、最小修复后重新加载与当前验证

```
systemctl daemon-reload
findmnt --verify
mount -a
findmnt <TARGET>
systemctl --failed
```

若 `mount -a` 报错，返回 Journal 和目标 unit，不要继续重启。若 `mount -a` 成功，也要确认 `findmnt <TARGET>` 的 source、fstype 和 options 与要求一致。

## 五、返回默认目标或进行受控重启

当前验证闭环后，可根据环境选择：

```
systemctl default
```

或在确认控制台与回退路径后正常重启。重启后：

```
systemctl get-default
systemctl --failed
findmnt <TARGET>
journalctl -b -p err
```

最终验收不是“系统离开 emergency”这一单点，而是系统能无人干预到达原 default target，目标文件系统按正确 source 持久挂载，且相关 mount unit 不再失败。

## 六、典型错误与修复

- 错误：直接注释失败行。修复：必需挂载必须修正真实原因，而不是隐藏。
- 错误：给必需挂载加 nofail。修复：只有业务明确允许设备缺失时才评估该选项。
- 错误：看到设备不挂载就执行 mkfs。修复：先通过 lsblk -f、blkid 和日志识别已有文件系统。
- 错误：mount -a 无输出后立即 reboot。修复：用 findmnt <TARGET> 和 systemctl --failed 分层验证，并始终保持最小变更，使故障原因和修复一一对应。

## [本章收束] 先判阶段、再判根、最后决定是否重启

启动与恢复题目表面上涉及 GRUB、target、rd.break、fstab 和 Journal，底层却共享同一条决策链：

```
症状出现在哪里
→ 当前处于哪个启动阶段
→ 当前 / 与真实根分别是什么
→ 目标状态是一次性还是持久
→ 这一阶段最有区分度的证据是什么
→ 做哪一个最小修复
→ 当前环境如何验证
→ 是否具备控制台和回退条件
→ 受控重启后如何用同一对象复验
```

## 工作方法：把恢复过程写成可回放的证据链

真正稳定的操作不是背下一串命令，而是让每一步都能回答三个问题：它正在改变哪个对象，下一条证据怎样证明它改变正确，以及这条证据还不能证明什么。grubby --info=ALL 可以证明启动条目文本，不能证明目标 kernel 已经成功启动；systemctl get-default 可以证明默认 target，不能证明当前系统已切换；mount -a 没有报错，也不能代替对目标挂载点和数据可见性的验证。

主要判断表

| 场景                              | 先查什么                                                           | 能证明什么         | 仍不能证明什么                |
|---------------------------------|----------------------------------------------------------------|---------------|------------------------|
| 当前 kernel 与参数                   | <code>uname -r</code> 、 <code>cat /proc/cmdline</code>         | 本次实际运行和收到的参数  | 下次默认条目是否相同             |
| 持久 kernel 参数                    | <code>grubby --info=ALL</code>                                 | 启动条目中的未来配置    | 条目能否成功完成启动             |
| default target                  | <code>systemctl get-default</code>                             | 正常启动入口        | 当前 active target 和业务功能 |
| <code>rd.break</code> 环境        | <code>cat /proc/cmdline</code> 、 <code>findmnt /sysroot</code> | 当前阶段与真实根位置    | 口令、标签和最终登录是否正确         |
| <code>fstab</code><br>emergency | <code>systemctl --failed</code> 、 <code>journalctl -b</code>   | 失败 unit 和时间序列 | 设备身份与配置字段谁错误           |
| SELinux 重标记                     | <code>touch /.autorelabel</code> 后正常启动                         | 已请求下一次完整重标记   | 重标记是否完成、模式和服务是否正常      |
| 重启后验收                           | 重查同一组对象并验证功能                                                   | 持久状态真正进入运行状态  | 所有外部业务都必然正确            |

向下一章交接

本章结束于“系统能够安全启动、进入预期 target，并能用证据确认恢复终态”。下一章《Podman 镜像、容器与 Rootless 运行》将在这个可用主机上引入容器镜像、容器实例和普通用户运行边界。容器启动故障仍可能需要 systemd 与 Journal 证据，但不会重新展开本章的 GRUB、initramfs 和系统恢复流程。

验证范围： 本章命令、参数和流程依据 RHEL 9 课程、官方文档与 man page 完成静态核对。固件按键时机、GRUB 菜单行为、`rd.break` 环境、SELinux 重标记耗时和重启结果仍需在目标 RHEL 9 主机上验证。